

ACADEMIC TRACK: CYBERSECURITY & CRYPTOGRAPHIC SYSTEMS

End-to-End Cryptographic Audit: Authenticated AES-256-GCM & HMAC-SHA-256

Submission Tracking ID: SUB-3B90E4 | Author: [Redacted for Double-Blind Evaluation] | Disclosure Status: Level 0 Anonymous

Abstract: Cryptographic audit ledgers require strict authenticity, confidentiality, and tamper-evident guarantees. This audit report investigates an enterprise peer assessment platform utilizing authenticated encryption (AES-256-GCM) alongside an append-only HMAC-SHA-256 cryptographic chain. We conduct threat modeling against STRIDE vectors, inspect nonce uniqueness guarantees under horizontal scaling, and verify sidecar isolation against cross-service privilege escalation.

1. Threat Modeling & Attack Surface Formulation

Academic peer grading environments are susceptible to grading collusion, selective disclosure extortion, and identity deanonymization. We evaluated the attack surface under the STRIDE methodology across six vectors.

2. Authenticated Encryption at Rest (AES-256-GCM)

All student submissions undergo authenticated encryption prior to filesystem persistence. A dedicated PHP 8.2+ cryptographic sidecar generates cryptographically secure 96-bit initialization vectors (IV) via random_bytes(12). The ciphertext is authenticated using a 128-bit authentication tag.

Vulnerability Domain	Cipher Suite / Control	STRIDE Vector	CVSS v3.1	Status	Residual Risk
Storage at Rest	AES-256-GCM (128-bit Tag)	Info Disclosure	7.5 (High)	Remediated	Negligible
Audit Log Integrity	HMAC-SHA-256 Chaining	Tampering	8.2 (High)	Remediated	Negligible
Reviewer Anonymity	Pseudonym Hashing	Info Disclosure	6.8 (Medium)	Remediated	Low
Identity Unmasking	Quorum Voting Threshold	Elevation Priv	8.9 (High)	Enforced	Zero

3. Tamper-Evident HMAC Audit Chain Verification

Every audit ledger transaction includes the previous event's cryptographic hash, forming an append-only hash chain: $H_i = \text{HMAC_SHA256}(\text{SecretKey}, H_{i-1} || \text{Action} || \text{Role} || \text{TargetId} || \text{Timestamp})$. Tampering with any historical block is detected immediately.

4. Remediation Strategy

We recommend rotating master keys periodically and storing HMAC roots in hardware security modules.